PROJETO QUIZ ARCOR CHALLENGE

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO E RA'S

Daniel Quio	26028696
Gisele Santos	26029631
João Souza	26029757
Murilo Pereira	26029666

São Paulo

2026

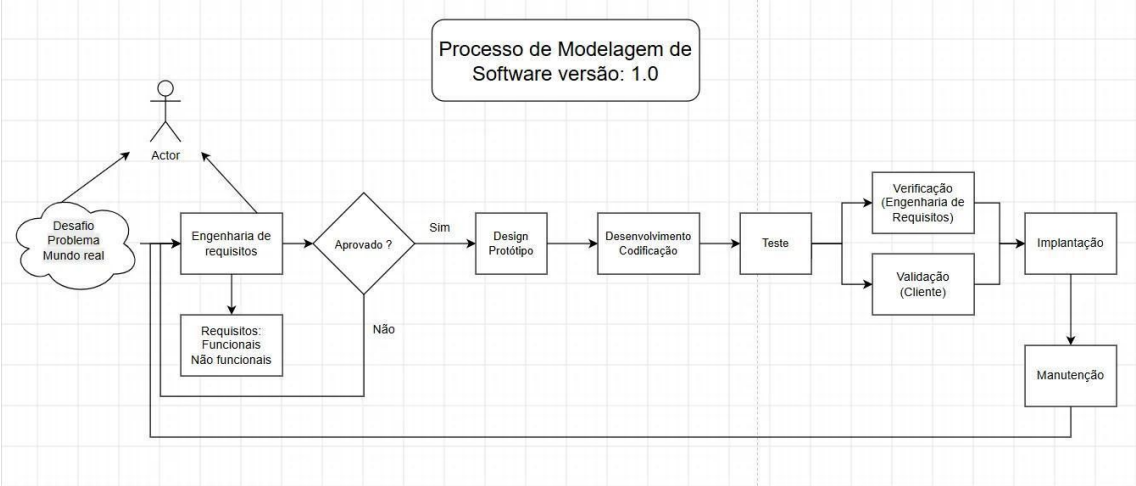
Sumário

1 INTRODUÇÃO....3
1.1 Processo de desenvolvimento de software....3
1.2 Apresentação....3
2 DESIGN SPRINT – IDEAÇÃO E PROTOTIPAÇÃO DO DESAFIO....4
2.1 Desafio....4
2.2 Entender e mapear....4
2.2.1 Entendimento e observação: COMO PODEMOS?....5
2.2.2 Pesquisa Desk....6
2.3 Ideação – desenho da solução (trilha do usuário)....9
2.4 Prototipagem....11
3 REQUISITOS DE SISTEMA....14
3.1 Requisitos funcionais de software....14
3.2 Requisitos não funcionais de software....16
4 DIAGRAMAS DE UML....18
4.1 Casos de uso....18
4.1.1 Diagrama geral de casos de uso – Jogador....18
4.1.2 Diagrama de casos de uso – Administrador....19
4.2 Diagrama de classe....20
5 ESPECIFICAÇÃO ESTRUTURADA DE UMA ROTINA DO SISTEMA....22
5.1 Especificação estruturada – Histórico de partidas....22
5.2 Codificação da rotina....22
5.2.1 Pseudocódigo....23
5.2.2 Código em Python....23
5.2.3 Implementação no sistema oficial (C#)....26
6 ARQUITETURA DO SISTEMA....27
6.1 Visão geral do sistema....27
6.2 Arquitetura geral....28
6.3 Componentes arquiteturais e camadas....28
6.3.1 Camada de apresentação (frontend)....28
6.3.2 Camada de aplicação (backend/API)....28
6.3.3 Camada de domínio (negócio)....29
6.3.4 Camada de persistência (banco de dados)....30
6.4 Diagrama da arquitetura....30
6.5 Organização de pastas e módulos....32
6.6 Integrações externas....32
6.7 Ambiente e implantação....32
7 REFERÊNCIAS....33

1 INTRODUÇÃO

1.1 Processo de desenvolvimento de software

Figura 1 – Processo de modelagem de software 1.0



Fonte: Elaborado pelos autores (2026).

1.2 Apresentação

O Projeto Integrador (PI) da FECAP tem como objetivo desenvolver uma experiência digital interativa voltada aos consumidores da marca Arcor, utilizando a tecnologia e a criatividade como ferramentas para promover entretenimento e engajamento com seus produtos.

Como proposta do projeto, nossa equipe irá desenvolver um jogo interativo baseado nos produtos Block e Tortuguita, buscando aproximar os consumidores da marca por meio de uma experiência divertida, dinâmica e competitiva.

O jogo contará com desafios e perguntas relacionados aos produtos, à marca Arcor e a suas características, utilizando um sistema de pontuação e níveis de dificuldade. Os participantes poderão avançar entre os níveis fácil, médio e difícil, acumulando pontos de acordo com seu desempenho.

A proposta é criar uma experiência que, além de entreter, incentive a interação dos consumidores com a marca Arcor e aumente o interesse pelos produtos Block e Tortuguita, transformando o conhecimento sobre a marca em uma atividade lúdica e envolvente.

Dessa forma, o projeto busca unir tecnologia, criatividade, entretenimento e interação com o consumidor, desenvolvendo uma solução digital capaz de proporcionar uma experiência diferenciada para o público da Arcor.

2 DESIGN SPRINT – IDEAÇÃO E PROTOTIPAÇÃO DO DESAFIO

2.1 Desafio

Proposta inicial do PI: nosso grupo irá desenvolver um quiz interativo sobre a marca Arcor, dos produtos Block e Tortuguita, com o objetivo de proporcionar uma experiência divertida e dinâmica, ao mesmo tempo em que os participantes conhecem mais sobre a empresa, seus produtos e sua história.

O quiz será dividido em três níveis de dificuldade: fácil, médio e difícil, permitindo que os participantes avancem de acordo com o nível de conhecimento. Cada pergunta terá uma pontuação de acordo com sua dificuldade.

Ao final, a pontuação de cada participante será contabilizada, permitindo a criação de um ranking para destacar os maiores resultados. Dessa forma, o projeto busca unir conhecimento, interação e competição saudável, tornando o aprendizado sobre a Arcor mais interessante e envolvente.

2.2 Entender e mapear

Nesta etapa, o grupo buscou compreender melhor o público que participa do quiz e as informações relacionadas à marca Arcor e aos produtos Block e Tortuguita. Foram consideradas características dos possíveis usuários, seus conhecimentos prévios sobre a empresa e os produtos, além da forma como uma atividade interativa poderia contribuir para aumentar o interesse e o envolvimento com a marca.

Também foram identificados os principais envolvidos no projeto, como os participantes do quiz, a marca Arcor e o grupo responsável pelo desenvolvimento da solução. A partir desse levantamento, foram observadas oportunidades para tornar a experiência mais dinâmica, simples e atrativa, utilizando perguntas, diferentes níveis de dificuldade e pontuação para estimular a participação.

Primeiramente, serão pesquisadas informações sobre a história da Arcor, seus produtos, características do Block e da Tortuguita e curiosidades relacionadas à marca. Essas informações serão utilizadas como base para a elaboração das perguntas e para definir os diferentes níveis de dificuldade do quiz.

Também serão considerados os possíveis usuários da solução, buscando entender seus conhecimentos prévios sobre a marca e quais elementos podem tornar a experiência mais atrativa. A partir disso, o grupo pretende utilizar perguntas objetivas, elementos visuais, diferentes níveis de dificuldade e um sistema de pontuação para estimular a participação.

Além disso, serão mapeados os principais envolvidos no projeto, considerando os participantes do quiz, a Arcor e o grupo responsável pelo desenvolvimento da solução. Com essas informações,

será possível identificar as principais necessidades e oportunidades para orientar a criação do protótipo.

Por fim, os dados levantados serão analisados pelo grupo e transformados em perguntas no formato “Como podemos...?”, permitindo identificar possíveis soluções para tornar o quiz mais interativo, informativo, dinâmico e divertido.

Tudo isso está intimamente relacionado ao senso de missão, visão e valores da Arcor, marca que tem como valores a proximidade com o consumidor e o compromisso com a cadeia de valor. O nosso foco principal é no público adulto e adolescente, trazendo a nostalgia e o apego da geração millennial e toda a energia da geração Z.

2.2.1 Entendimento e observação: COMO PODEMOS?

1. Como podemos criar uma experiência imersiva para o nosso público-alvo? Primeiramente, o que já sabemos sobre o usuário?
2. Como podemos tornar o quiz mais divertido e interessante?
3. Como podemos despertar a curiosidade dos participantes sobre a história da Arcor?
4. Como podemos utilizar perguntas criativas para incentivar a participação?
5. Como podemos criar níveis de dificuldade que proporcionem desafios diferentes aos participantes?
6. Como podemos motivar os participantes a avançarem entre os níveis do quiz?
7. Como podemos utilizar a pontuação para aumentar o interesse dos participantes?
8. Como podemos criar um ranking que estimule uma competição saudável?
9. Como podemos evitar que o quiz se torne repetitivo ou cansativo?
10. Como podemos utilizar imagens e elementos visuais para deixar o quiz mais atrativo?
11. Como podemos criar uma experiência interessante para pessoas que já conhecem a Arcor e para aquelas que conhecem pouco sobre a marca?
12. Como podemos transformar informações sobre a Arcor em perguntas que sejam fáceis de compreender?
13. Como podemos unir entretenimento e aprendizado durante a realização do quiz?
14. Como podemos desenvolver um quiz interativo que proporcione uma experiência divertida e, ao mesmo tempo, aumente o conhecimento dos participantes sobre a Arcor, o Block e a Tortuguita?

2.2.2 Pesquisa Desk

Mercado e consumo de chocolate

O aumento do preço do cacau vem pressionando o mercado de chocolates e impactando o comportamento do consumidor. Segundo a Kantar, o consumo domiciliar de chocolate no Brasil caiu 19% em volume no primeiro trimestre de 2025, em um cenário de forte alta da matéria-prima. Esse movimento coloca a categoria diante de um desafio: com o chocolate mais caro, manter o interesse e a conexão do consumidor com as marcas se torna ainda mais relevante.

Fonte: Kantar, 2025 — "Queda no consumo de chocolate em meio à crise do cacau".

Gamificação e aprendizagem

A gamificação utiliza elementos de jogos, como pontuação, desafios, competição e rankings, em atividades que não são necessariamente jogos. Estudos apontam que esses recursos podem contribuir para o engajamento, a motivação e a participação dos usuários. Para o projeto, isso reforça a possibilidade de utilizar elementos de gamificação para transformar o quiz da Arcor em uma experiência mais dinâmica e envolvente.

Fonte: Zainuddin et al., 2020 — "The impact of gamification on learning and instruction: A systematic review of empirical evidence".

Gamificação e motivação

Pesquisas sobre gamificação indicam que elementos como pontos, medalhas e rankings podem influenciar a motivação dos participantes. Entretanto, os resultados dependem da maneira como esses recursos são utilizados e do contexto da atividade. Para o quiz, esses elementos poderão ser utilizados para estimular a participação sem tornar a experiência excessivamente competitiva.

Fonte: Sailer et al., 2017 — "How gamification motivates: An experimental study of the effects of specific game design elements on psychological need satisfaction".

Quizzes como ferramenta de aprendizagem

Os quizzes digitais podem ser utilizados para testar conhecimentos, revisar conteúdos e aumentar a participação dos usuários. Uma revisão que analisou 93 estudos sobre o Kahoot! identificou resultados positivos relacionados à aprendizagem, participação e percepção dos estudantes, mas também apontou desafios relacionados à pressão de tempo e à dificuldade das perguntas. Esses fatores devem ser considerados na criação do quiz da Arcor.

Fonte: Wang; Tahir, 2020 — "The effect of using Kahoot! for learning – A literature review".

Competição e ranking

O uso de rankings é comum em experiências gamificadas e permite que os participantes acompanhem seu desempenho em relação aos demais. Apesar de poder estimular a participação, a competição precisa ser equilibrada para evitar pressão ou frustração. No projeto, o ranking será

utilizado como forma de incentivar os participantes a melhorar seus resultados e continuar interagindo com o quiz.

Fonte: Sailer et al., 2017 — "How gamification motivates: An experimental study of the effects of specific game design elements on psychological need satisfaction".

Feedback

O feedback imediato é um elemento importante em experiências interativas, pois permite que o usuário compreenda o resultado de suas ações. Em um quiz, informar se a resposta está correta ou incorreta logo após a escolha pode transformar o erro em uma oportunidade de aprendizado. Por isso, o projeto poderá apresentar uma explicação ou curiosidade após cada resposta.

Fonte: Hong; Saab; Admiraal, 2024 — "Approaches and game elements used to tailor digital gamification for learning: A systematic literature review".

Tempo de resposta

Plataformas de quiz como Kahoot! utilizam o tempo de resposta como um dos elementos da experiência. Embora um cronômetro possa aumentar o desafio e tornar a atividade mais dinâmica, pesquisas também apontam que a pressão de tempo pode prejudicar a experiência de alguns participantes. Por isso, o uso do cronômetro deverá ser avaliado durante os testes do projeto.

Fonte: Wang; Tahir, 2020 — "The effect of using Kahoot! for learning – A literature review".

Níveis de dificuldade

A divisão das perguntas em diferentes níveis de dificuldade pode criar uma sensação de progressão para o participante. No projeto, os níveis fácil, médio e difícil serão utilizados para organizar as perguntas de acordo com o conhecimento necessário, permitindo também estabelecer diferentes valores de pontuação para cada etapa.

Fonte: Pesquisa Desk do grupo, 2026 — análise de referências de plataformas de quizzes e gamificação.

Kahoot! como referência

O Kahoot! utiliza perguntas, respostas, pontuação, competição e feedback para criar experiências interativas de aprendizagem. A plataforma serve como referência para o projeto por apresentar uma estrutura semelhante à proposta do grupo, demonstrando como conteúdos informativos podem ser transformados em uma experiência dinâmica e participativa.

Fonte: Wang; Tahir, 2020 — "The effect of using Kahoot! for learning – A literature review".

Quizizz como referência

O Quizizz utiliza quizzes digitais associados a elementos de gamificação para estimular a participação dos usuários. A plataforma demonstra como perguntas e conteúdos educativos podem ser apresentados de maneira mais interativa, servindo como referência para a estrutura e dinâmica do quiz sobre a Arcor.

Fonte: Quizizz, 2026 — Plataforma de aprendizagem gamificada.

Mentimeter como referência

O Mentimeter possui um sistema de competição em quizzes no qual os participantes respondem às perguntas, recebem pontuação e acompanham sua posição em um ranking. Essa estrutura apresenta semelhanças com a proposta do grupo e serve como referência para o fluxo do usuário: entrar, informar o nome, responder às perguntas, receber a pontuação e visualizar o resultado final.

Fonte: Mentimeter, 2026 — "How to create a quiz competition".

Mercado de chocolates

O mercado de chocolates possui diversas marcas e produtos que disputam a atenção dos consumidores. A Arcor atua no segmento de chocolates desde 1970 e possui marcas como Tortuguita, Bon o Bon, Cofler, Tofi, Águila e Rocklets. Esse cenário demonstra a importância de trabalhar não apenas o produto, mas também a relação e a comunicação da marca com o consumidor.

Fonte: Arcor, 2026 — Informações institucionais sobre o negócio de chocolates.

Concorrentes

Grandes empresas como a Nestlé possuem um amplo portfólio de chocolates no mercado brasileiro, incluindo marcas como KitKat, Talento, Crunch, Suflair, Alpino e Prestígio. A análise dessas marcas permite compreender o cenário competitivo do segmento e observar como diferentes empresas trabalham seus produtos e sua comunicação com os consumidores.

Fonte: Nestlé Brasil, 2026 — "Chocolates".

Arcor, Block e Tortuguita

A Arcor possui um portfólio diversificado de chocolates e apresenta a Tortuguita entre suas principais marcas. Essas informações serão utilizadas como base para a criação das perguntas do quiz, permitindo abordar a história da empresa, seus produtos, características das marcas e curiosidades relacionadas ao universo da Arcor.

Fonte: Arcor, 2026 — Informações institucionais e portfólio de chocolates.

Oportunidade identificada

A pesquisa demonstra uma oportunidade de unir informação, entretenimento e gamificação em uma única experiência. Em vez de apresentar informações sobre a Arcor, Block e Tortuguita apenas por meio de textos, o projeto poderá transformar esse conteúdo em uma experiência na qual o participante responde perguntas, recebe feedback, acumula pontos, avança pelos níveis e acompanha seu resultado.

Fonte: Pesquisa Desk do grupo, 2026 — Síntese das pesquisas realizadas sobre gamificação, quizzes e mercado de chocolates.

2.3 Ideação – desenho da solução (trilha do usuário)

No projeto em questão, o usuário passará por etapas de cadastro, validação de login e senha, escolha da dificuldade do jogo, o jogo em si, o ranking em relação a outros usuários e a possibilidade de jogar quantas vezes quiser, convidar amigos e tornar a experiência satisfatória ao nosso público.

Para isso, a Pesquisa Desk foi utilizada como uma das etapas de investigação para compreender o comportamento dos consumidores e a relação estabelecida entre marcas de chocolate e seus públicos. A partir da análise de fontes secundárias, foi possível observar que determinadas marcas podem ultrapassar a relação de consumo do produto e passar a fazer parte do imaginário, das tradições e dos momentos marcantes dos consumidores.

Nesse sentido, uma reportagem da Folha de S.Paulo, ao abordar o consumo de chocolate e a presença das marcas em momentos específicos, apresenta a perspectiva de Marcos Bedendo, professor de branding e marketing da ESPM, de que as marcas que conseguem se inserir no imaginário e nas tradições dos consumidores podem tornar-se parte de seus rituais.

Essa perspectiva é relevante para o desenvolvimento do quiz da Arcor, uma vez que o projeto trabalha com dois produtos e públicos distintos: Tortuguita, direcionado a um público mais velho, e Block, direcionado à Geração Z. A pesquisa permite considerar não apenas as características dos produtos, mas também aspectos relacionados à memória, identificação, linguagem e experiências associadas às marcas.

Dessa forma, a Pesquisa Desk contribui para a definição das características do quiz, auxiliando a equipe a compreender quais elementos podem gerar identificação com cada público e como adaptar a experiência digital às diferentes relações que os consumidores estabelecem com as marcas.

Pensando nisso, pode-se pensar em problemas e em como solucioná-los, por exemplo: e se eu tivesse superpoderes? Como poderia escalar esta aplicação de forma quase impensável? Talvez um complexo de jogos dentro de um aplicativo, sendo um jogo para cada item da Arcor. Ou mesmo, “Se eu tivesse muito dinheiro”, escalar a nossa aplicação para utilizar recursos em nuvem com instâncias AWS, tornando o jogo escalável e acessado de qualquer lugar do globo. Ou até “E se eu não tivesse dinheiro algum?”: qual o menor produto viável de um quiz interativo? E, por fim, “E se eu fosse o Dick Vigarista e quisesse boicotar o projeto?”: possivelmente o Dick Vigarista forçaria um ataque ao nosso banco de dados e tentaria burlar as regras do nosso sistema. Estaríamos preparados para isso? Essas questões são tratadas nos requisitos de segurança (seção 3.2) e na arquitetura do sistema (seção 6).

Já pensando em ideias inacessíveis, podemos listar oito ideias que seriam quase impossíveis de tornar realidade no projeto, como:

1. O jogador que conseguir uma sequência de 10 vitórias no Quiz ganhará 1 ano de produtos Arcor.

2. A cada vez que o jogador compartilhar o jogo com um amigo, ganhará de brinde um ovo de Páscoa de 1 kg.

3. Em nosso jogo, terá uma tartaruga interativa que falará com nosso usuário da forma que ele solicitar.

4. Cada pessoa que tentar responder ao quiz ganha 200 reais.

5. Nosso quiz portará um sistema de rastreio para a residência do usuário, para que a Arcor envie produtos mensalmente aos jogadores mais presentes no jogo.

6. Em vez de um prêmio convencional, o participante concorre a algo completamente inesperado: ganhar um cupom de 1.000 reais.

7. Ganhar o prêmio de visitar a fábrica e entender como ela funciona.

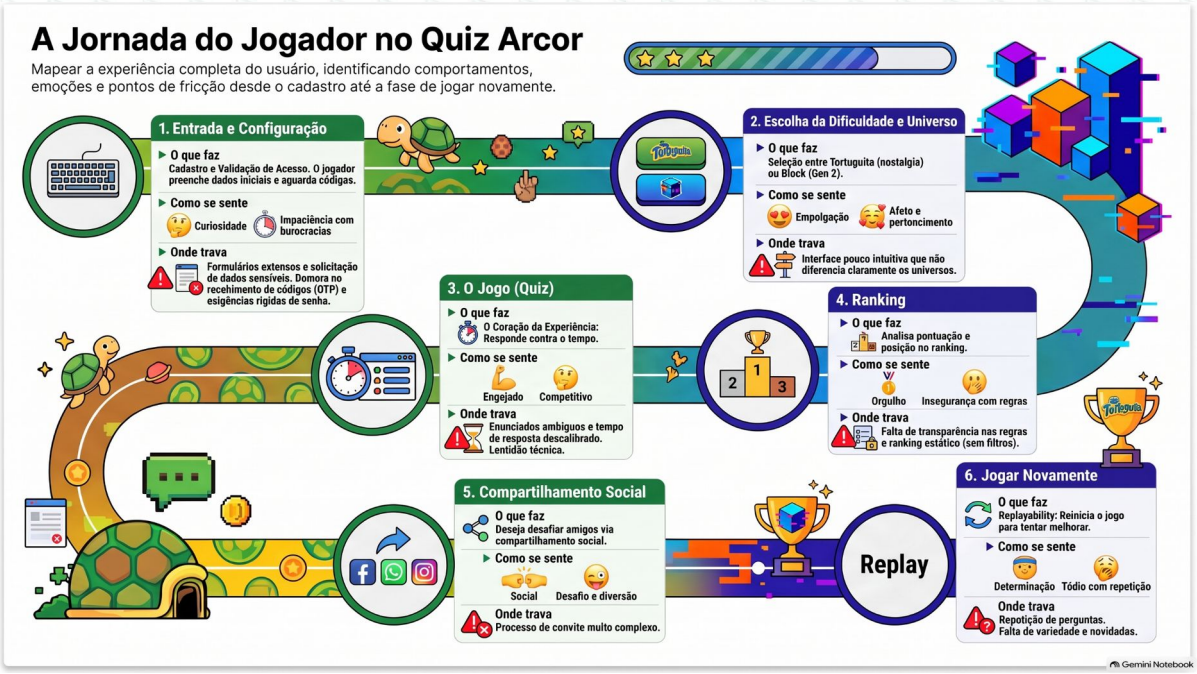
8. Uma Tortuguita está escondida. Quem encontrar primeiro ganha R$ 500.

Quadro 1 – Trilha do usuário no Quiz

Etapa	Descobrir	Escolher	Jogar	Ver resultado
O que o aluno faz	Abre o Quiz	Modo de jogo: Fácil, Normal ou Difícil	Joga nosso quiz	Olha quantos acertos obteve
Como ele se sente	Disposto a aprender mais sobre a marca	Em dúvida	Competitivo	Satisfeito ou insatisfeito
Onde ele trava	Cadastro	“Sou bom o bastante?”	Perguntas difíceis	Não saber onde errou

Fonte: Elaborado pelos autores (2026).

Figura 2 – A jornada do jogador no Quiz Arcor



Fonte: Elaborado pelos autores (2026).

2.4 Prototipagem

A partir da trilha do usuário, o grupo elaborou o protótipo das principais telas do Quiz. A tela inicial apresenta a identidade da Arcor e o convite para jogar; em seguida, o jogador escolhe o modo de jogo, responde às perguntas com o contador de progresso e, ao final, visualiza o resultado com as opções de compartilhar e jogar novamente.

Figura 3 – Protótipo da tela inicial e da seção “Sobre nós”



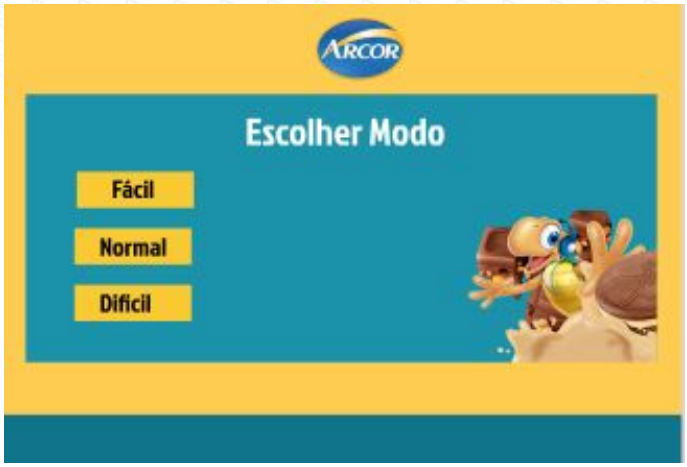
Fonte: Elaborado pelos autores (2026).

Figura 4 – Protótipo da tela de abertura do desafio Tortu-Block



Fonte: Elaborado pelos autores (2026).

Figura 5 – Protótipo da tela de escolha do modo de jogo



Fonte: Elaborado pelos autores (2026).

Figura 6 – Protótipo da tela de pergunta



Fonte: Elaborado pelos autores (2026).

Figura 7 – Protótipo da tela de resultado



Fonte: Elaborado pelos autores (2026).

3 REQUISITOS DE SISTEMA

Os requisitos abaixo foram escritos a partir do funcionamento da versão oficial do sistema, composta pelo aplicativo QuizArcor (C#, Windows Forms), pela API QuizArcorApi (C#, ASP.NET Core) e pelo banco de dados SQLite quiz.db.

3.1 Requisitos funcionais de software

RFS01	
Função	Tela inicial, instruções e identificação do jogador
Descrição	Apresentar a identidade do projeto, o objetivo do jogo, as instruções de uso (“Como jogar”) e as opções para iniciar uma partida ou consultar o histórico. Antes de jogar, o jogador informa um apelido (nickname) ou escolhe jogar como anônimo.
Entradas	Apelido (nickname) ou opção “Jogar como anônimo”
Fonte	Jogador, pela tela “Quem vai jogar?” do aplicativo
Saídas	Jogador identificado e tela de escolha da dificuldade
Ação	O sistema exibe a capa com os botões “Como jogar”, “JOGAR” e “Histórico”. Ao clicar em “JOGAR”, o jogador digita o apelido (até 30 caracteres) ou marca “Jogar como anônimo”; se o campo ficar vazio, o nome “Anônimo” é usado. Nenhum dado pessoal, como e-mail, senha ou faixa etária, é solicitado.

RFS02	
Função	Carga de conteúdos do banco de dados
Descrição	Recuperar perguntas, alternativas, pistas, feedbacks e informações das marcas diretamente do banco de dados relacional, por meio da API.
Entradas	Consulta ao banco de dados (GET /api/perguntas e GET /api/marcas)
Fonte	Banco de dados SQLite (quiz.db), tabelas Perguntas e Marcas
Saídas	Lista de perguntas em JSON, cada uma com quatro alternativas (A a D), resposta correta, dificuldade, pista, feedback e marca
Ação	Cada pergunta está armazenada em banco de dados relacional, pois precisa dizer respeito exatamente ao que o cliente disponibiliza de informação. A API sorteia as perguntas ativas (ORDER BY RANDOM), aplica os filtros de quantidade, dificuldade e marca e pode excluir perguntas já usadas na partida, evitando repetição.

RFS03	
Função	Escolha da dificuldade
Descrição	Permitir que o jogador escolha a dificuldade das perguntas: Todas, Fácil, Médio ou Difícil.
Entradas	Dificuldade escolhida
Fonte	Jogador, pela tela de identificação do aplicativo
Saídas	Conjunto de 10 perguntas da dificuldade escolhida e início da partida
Ação	O aplicativo solicita 10 perguntas à API com o filtro de dificuldade (ou sem filtro, na opção “Todas”). Perguntas com dados inconsistentes são descartadas. Se não houver perguntas disponíveis ou a API estiver fora do ar, o jogador recebe uma mensagem e a partida não é iniciada.

RFS04	
Função	Partida do Quiz: perguntas, tempo, pista e feedback
Descrição	Exibir as perguntas uma a uma, com quatro alternativas coloridas, contador de progresso (“Pergunta 3 de 10”), marca, pontos e um tempo de 20 segundos por pergunta. O jogador pode abrir uma pista e, após responder, recebe um feedback educativo.
Entradas	Alternativa escolhida (clique ou teclas 1 a 4 / A a D) e pedido de pista
Fonte	Jogador; perguntas, pistas, feedbacks e descrições das marcas vindos da API
Saídas	Indicação de acerto ou erro, resposta correta, feedback, descrição da marca e próxima pergunta
Ação	O sistema compara a letra escolhida com a resposta correta. Se o tempo acabar, a questão conta como erro. Ao acertar, a tela pisca em verde e o mascote aparece feliz; ao errar, pisca em vermelho e mostra a resposta correta. A pista custa 5 pontos e é cobrada apenas na primeira vez que é aberta em cada pergunta. Nas perguntas da Block, o fundo muda para o tema chocolate.

RFS05	
Função	Pontuação, classificação e resultado
Descrição	Calcular a pontuação de acordo com a dificuldade e exibir o resumo final da partida, com classificação do jogador.
Entradas	Acertos por nível, pistas usadas e tempo da partida
Fonte	Classe CalculadoraPartida do aplicativo
Saídas	Tela de resultado com troféu, classificação, pontos e resumo detalhado
Ação	Cada acerto vale 10 pontos (Fácil), 20 (Médio) ou 30 (Difícil); cada pista desconta 5 pontos e a pontuação nunca fica negativa. O resumo mostra acertos e percentual por nível, pontuação máxima e aproveitamento, tempo médio por questão, ritmo (Rápido, Normal ou Pausado), melhor nível e a classificação: Mestre das Marcas (90% ou mais), Conhecedor de Marcas (70%), Aprendiz (50%) ou Iniciante.

RFS06	
Função	Histórico de partidas
Descrição	Registrar automaticamente o resultado da partida ao seu término e permitir consultar as partidas registradas, exibindo as mais recentes primeiro.
Entradas	Nickname, pontuação, modo de jogo, total de questões, total de acertos, tempo da partida e dicas usadas
Fonte	Resultado final da partida gerado pelo aplicativo do Quiz Arcor
Saídas	Lista com os resultados das partidas registradas (data/hora, jogador, pontos, acertos e tempo)
Ação	Ao finalizar, o aplicativo envia o resultado para POST /api/partidas, que grava na tabela Partidas com data e hora. Se o envio falhar, o botão “Tentar registrar de novo” é exibido. A tela “Histórico” consulta GET /api/partidas e mostra as últimas 20 partidas. A especificação completa está na seção 5.

3.2 Requisitos não funcionais de software

RNF01	
Função	Desempenho do Quiz
Descrição	O sistema deverá apresentar as perguntas, alternativas, pistas e resultados de forma rápida durante a partida.
Entradas	Solicitações do usuário
Fonte	Banco de dados SQLite
Saídas	Perguntas e informações do Quiz
Ação	As 10 perguntas da partida são carregadas de uma só vez no início, e as imagens de fundo são preparadas uma única vez e guardadas em memória. As chamadas à API têm tempo limite de 10 segundos e a consulta de perguntas é limitada a 50 itens, evitando esperas longas.

RNF02	
Função	Segurança dos dados
Descrição	Garantir que o banco de dados e as informações do Quiz sejam protegidos contra acessos e alterações indevidas.
Entradas	Parâmetros enviados à API (filtros e dados da partida)
Fonte	API QuizArcorApi e banco de dados SQLite
Saídas	Informações armazenadas com segurança
Ação	Todas as consultas usam parâmetros ($dif, $marca, $nick etc.), o que impede ataques de injeção de SQL. Erros inesperados viram uma resposta JSON, sem derrubar a API. O jogo não coleta dados pessoais (apenas o apelido, que pode ser anônimo), reduzindo o risco em caso de vazamento.

RNF03	
Função	Usabilidade da interface
Descrição	A interface deverá ser simples e intuitiva, com instruções claras, botões grandes, cores fortes e a identidade visual da Arcor.
Entradas	Interações do usuário com as telas
Fonte	Telas do aplicativo QuizArcor
Saídas	Navegação clara e sem dúvidas
Ação	O jogador inicia uma partida em poucos cliques. As alternativas têm cores e símbolos diferentes (triângulo, losango, círculo e quadrado), o tempo aparece em uma barra que muda de verde para amarelo e vermelho, há atalhos de teclado e o retorno visual de acerto ou erro é imediato.

RNF04	
Função	Compatibilidade e portabilidade
Descrição	O sistema deverá funcionar em computadores com Windows que tenham o .NET 10 instalado.
Entradas	Execução do aplicativo e da API
Fonte	Computador do usuário
Saídas	Janela do jogo em 1152 × 648 pixels
Ação	O aplicativo é desenvolvido em Windows Forms (net10.0-windows) e a API em ASP.NET Core (net10.0), que também pode rodar em outros sistemas operacionais. Se uma imagem não for encontrada, o jogo continua funcionando com um fundo em degradê.

RNF05	
Função	Confiabilidade e integridade dos dados
Descrição	Garantir que pontuações, histórico e conteúdo sejam gravados de forma correta e consistente.
Entradas	Dados das partidas e do conteúdo
Fonte	Aplicativo QuizArcor e API QuizArcorApi
Saídas	Registros consistentes no banco de dados
Ação	O aplicativo valida os dados da partida antes de calcular o resultado (acertos, tempo e quantidade de pistas) e descarta perguntas inconsistentes. A API recusa pontuação negativa e, ao iniciar, confere se as tabelas existem e acrescenta as colunas que faltarem.

RNF06	
Função	Manutenibilidade
Descrição	O sistema deverá ser organizado em camadas e manter o conteúdo no banco de dados, e não fixo no código.
Entradas	Alterações de conteúdo ou de código
Fonte	Equipe de desenvolvimento e administrador
Saídas	Sistema atualizado sem retrabalho
Ação	Aplicativo, API e banco são separados. Perguntas e marcas são cadastradas pelo script quiz.sql ou pelo DB Browser for SQLite, e podem ser inativadas pela coluna Ativo sem serem apagadas, sem alterar o código-fonte.

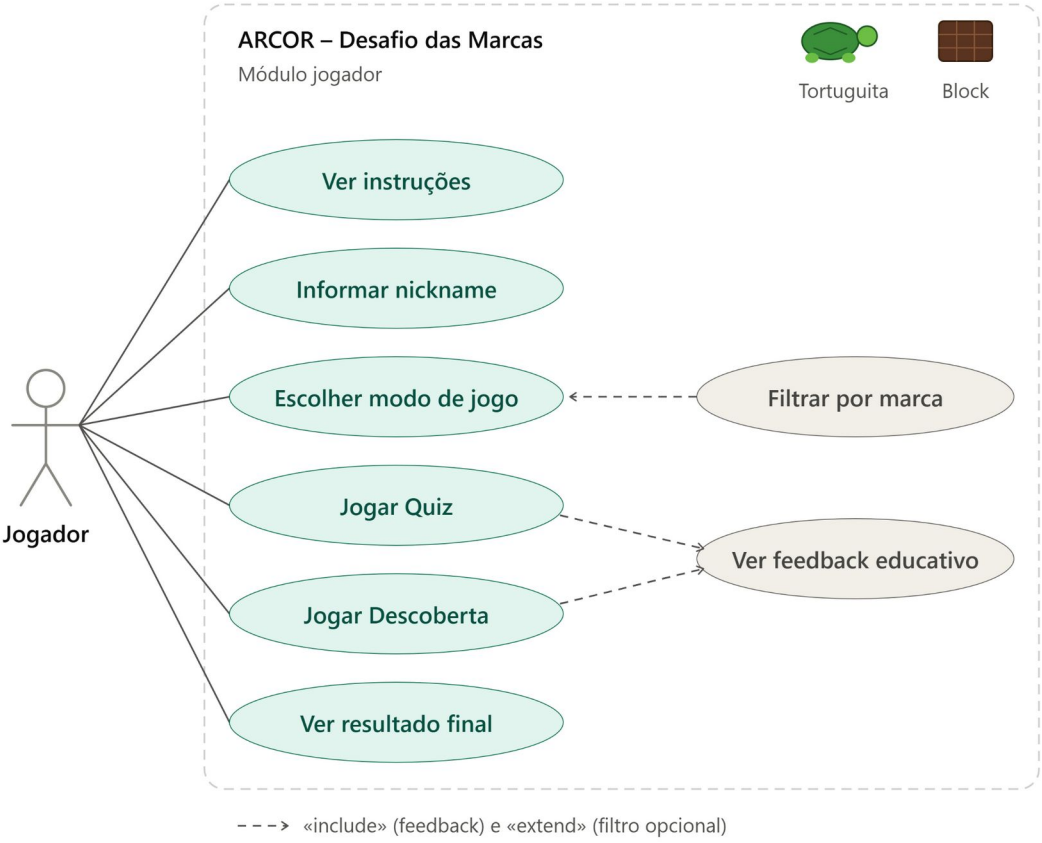
4 DIAGRAMAS DE UML

4.1 Casos de uso

4.1.1 Diagrama geral de casos de uso – Jogador

Este diagrama apresenta a visão geral do sistema, conectando os atores principais às suas respectivas funcionalidades. Ele detalha o fluxo de jogo do usuário (Geração Z e millennials), desde a identificação por nickname até a escolha dos modos e das marcas Tortuguita e Block (UC01 – Jogar Partida, modo Quiz ou Descoberta).

Figura 8 – Diagrama de casos de uso do módulo Jogador

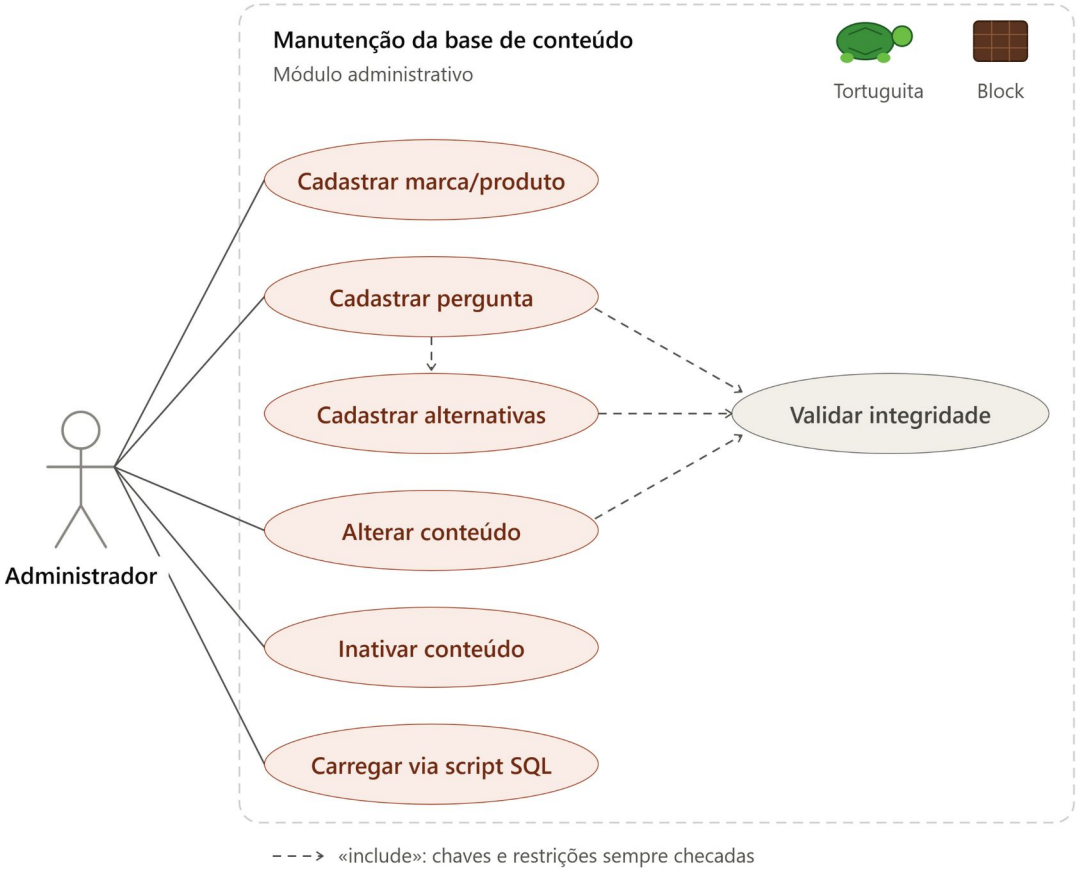


Fonte: Elaborado pelos autores (2026).

4.1.2 Diagrama de casos de uso – Administrador

O segundo diagrama é o lado do Administrador, que cadastra e mantém as perguntas de Tortuguita e Block no banco. Isso é o que comprova que o conteúdo não está fixo no código.

Figura 9 – Diagrama de casos de uso do módulo Administrador

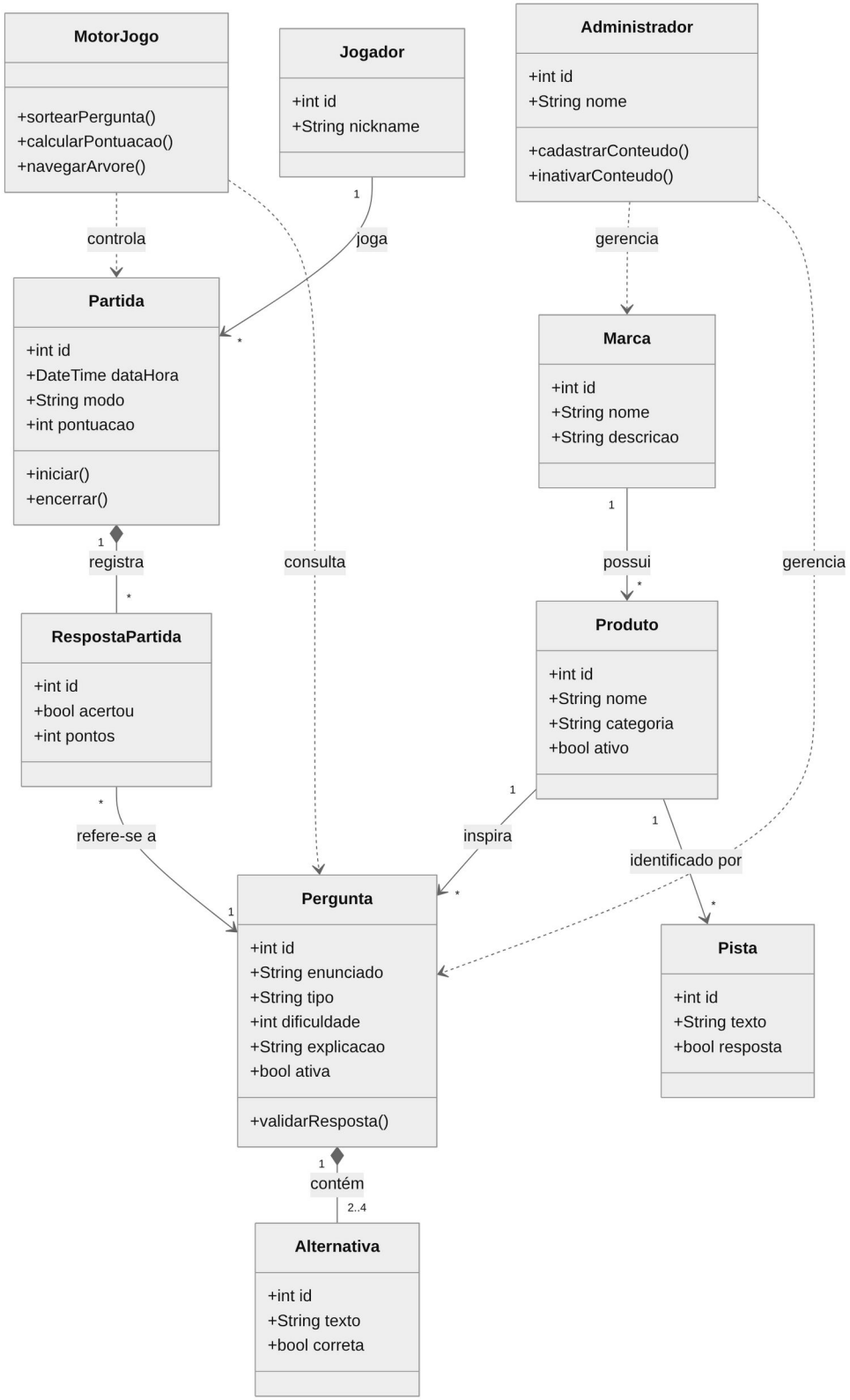


Fonte: Elaborado pelos autores (2026).

4.2 Diagrama de classe

O diagrama de classe apresenta as entidades do Quiz e seus relacionamentos. O Jogador joga várias Partidas, e cada Partida registra as respostas dadas (RespostaPartida), cada uma referente a uma Pergunta. A Pergunta contém de 2 a 4 Alternativas e é inspirada em um Produto, que pertence a uma Marca e é identificado por Pistas. O MotorJogo controla a partida e consulta as perguntas, enquanto o Administrador gerencia marcas e perguntas. O diagrama representa o modelo conceitual do sistema: na implementação oficial, as quatro alternativas e a pista são guardadas como colunas da tabela Perguntas (AlternativaA a AlternativaD, RespostaCorreta e Pista), e o resultado da partida é gravado na tabela Partidas.

Figura 10 – Diagrama de classe do Quiz Arcor Challenge



Fonte: Elaborado pelos autores (2026).

5 ESPECIFICAÇÃO ESTRUTURADA DE UMA ROTINA DO SISTEMA

5.1 Especificação estruturada – Histórico de partidas

Nome do requisito: Histórico.

Requisito funcional (problema): o sistema deverá registrar automaticamente o resultado da partida ao seu término, armazenando no banco de dados o nickname do jogador, a pontuação, o modo de jogo, a quantidade total de questões, o número de acertos, o tempo de duração e a quantidade de dicas utilizadas. O sistema também deverá permitir consultar as partidas registradas, exibindo as mais recentes primeiro.

Quadro 2 – Especificação estruturada do requisito Histórico

Campo	Conteúdo
Identificador	Projeto6\src\Entrega 2\Backend\QuizArcorApi\Historico
Função	Registrar e consultar o histórico das partidas.
Descrição	Recebe os dados salvos na partida e guarda no banco de dados; o sistema permite visualizar partidas passadas.
Entradas	Nickname, Pontuação, Pontos, Total de Acertos, Tempo da Partida e Data/Hora
Fonte	Resultado final da partida gerado pelo frontend do Quiz Arcor.
Saídas	Lista contendo os resultados das partidas registradas.
Destino	Banco de dados SQLite, na tabela Partidas, e frontend do sistema, que recebe os dados da consulta.
Ação	Inserir os dados da partida no banco de dados ao finalizar o jogo e disponibilizar os registros por meio de uma consulta ao histórico. Verificar se há pontuação; se não há, não fazer o registro. Fazer o registro apenas se todas as questões forem respondidas.
Requisitos	O banco de dados deve conter a tabela Partidas e os campos necessários. A pontuação não pode ser negativa. O sistema deve registrar a data e a hora da partida.
Pré-condição	Nickname ou modo anônimo; o jogador tem que escolher o modo de jogo e responder todas as questões.
Pós-condições	O sistema retornará os registros encontrados, ordenados do mais recente para o mais antigo.
Efeitos colaterais	Não se aplica.

Fonte: Elaborado pelos autores (2026).

5.2 Codificação da rotina

A rotina foi codificada a partir da especificação estruturada do Quadro 2, usando o mesmo banco (quiz.db), a mesma tabela Partidas e os mesmos nomes de colunas da API oficial. Cada regra da especificação aparece como uma verificação no código: nickname vazio vira “Anônimo”, modo vazio vira “Quiz”, a partida só é registrada se todas as questões forem respondidas e houver pontuação, a pontuação não pode ser negativa e a data e a hora são gravadas automaticamente. A consulta devolve as partidas da mais recente para a mais antiga.

5.2.1 Pseudocódigo

```
INÍCIO registrarPartida(nickname, pontuacao, modo, totalQuestoes,
                        respondidas, totalAcertos, tempoSegundos, dicasUsadas)
    SE nickname estiver vazio ENTÃO
        nickname ← "Anônimo"
    FIM SE
    SE modo estiver vazio ENTÃO
        modo ← "Quiz"
    FIM SE
    SE modo NÃO pertence a {Quiz, Descoberta} ENTÃO
        RETORNAR "Modo de jogo inválido"
    FIM SE
    SE respondidas ≠ totalQuestoes ENTÃO
        RETORNAR "Partida incompleta – registro não realizado"
    FIM SE
    SE pontuacao < 0 ENTÃO
        RETORNAR "Pontuação inválida"
    FIM SE
    SE pontuacao = 0 ENTÃO
        RETORNAR "Partida sem pontuação – registro não realizado"
    FIM SE
    INSERIR em Partidas (Nickname, Pontuacao, Modo, TotalQuestoes, TotalAcertos,
                         TempoSegundos, DicasUsadas, DataHora ← data e hora atuais)
    RETORNAR "Partida registrada"
FIM

INÍCIO consultarHistorico(limite)
    limite ← valor entre 1 e 200 (padrão 20)
    registros ← SELECIONAR as colunas de Partidas
                ORDENAR POR DataHora DECRESCENTE, Id DECRESCENTE
                LIMITAR A limite registros
    RETORNAR registros
FIM
```

5.2.2 Código em Python

O código abaixo usa o módulo sqlite3, que já vem com o Python. A função preparar_tabela acrescenta as colunas de métricas que faltarem, da mesma forma que a função PrepararBanco da API.

```
"""Rotina de Histórico de partidas - Quiz Arcor Challenge.

Registra o resultado da partida ao final do jogo e permite consultar
as partidas registradas, das mais recentes para as mais antigas.
Usa o mesmo banco SQLite (quiz.db) e a mesma tabela Partidas da
QuizArcorApi, com os mesmos nomes de colunas.
"""
import sqlite3

BANCO = "quiz.db"
MODOS_VALIDOS = ("Quiz", "Descoberta")
```

```
# Colunas de métricas que a API também acrescenta, se ainda não existirem
COLUNAS_HISTORICO = {
    "Modo": "TEXT NOT NULL DEFAULT 'Quiz'",
    "TotalQuestoes": "INTEGER NOT NULL DEFAULT 0",
    "TotalAcertos": "INTEGER NOT NULL DEFAULT 0",
    "TempoSegundos": "INTEGER NOT NULL DEFAULT 0",
    "DicasUsadas": "INTEGER NOT NULL DEFAULT 0",
}


def conectar(caminho=BANCO):
    conexao = sqlite3.connect(caminho)
    conexao.row_factory = sqlite3.Row
    preparar_tabela(conexao)
    return conexao


def preparar_tabela(conexao):
    """Requisito: o banco deve conter a tabela Partidas e os campos necessários."""
    conexao.execute(
        """CREATE TABLE IF NOT EXISTS Partidas (
               Id INTEGER PRIMARY KEY AUTOINCREMENT,
               Nickname TEXT NOT NULL,
               Pontuacao INTEGER NOT NULL,
               DataHora DATETIME DEFAULT CURRENT_TIMESTAMP
           )"""
    )
    existentes = {linha[1] for linha in conexao.execute("PRAGMA table_info(Partidas)")}
    for coluna, definicao in COLUNAS_HISTORICO.items():
        if coluna not in existentes:
            conexao.execute(f"ALTER TABLE Partidas ADD COLUMN {coluna} {definicao}")
    conexao.commit()


def registrar_partida(conexao, nickname, pontuacao, modo, total_questoes,
                      respondidas, total_acertos, tempo_segundos, dicas_usadas):
    """Grava a partida e devolve (True, mensagem) ou (False, motivo)."""
    # Pré-condição: nickname ou modo anônimo
    nickname = (nickname or "").strip() or "Anônimo"

    # Pré-condição: o modo de jogo precisa ser válido (padrão "Quiz", como na API)
    modo = (modo or "").strip() or "Quiz"
    if modo not in MODOS_VALIDOS:
        return False, "Modo de jogo inválido."

    # Ação: registrar apenas se todas as questões forem respondidas
    if total_questoes < 1 or respondidas != total_questoes:
        return False, "Partida incompleta: nem todas as questões foram respondidas."

    # Requisito: a pontuação não pode ser negativa
    if pontuacao < 0:
        return False, "Pontuação inválida."

    # Ação: verificar se há pontuação; se não há, não fazer o registro
```

```
    if pontuacao == 0:
        return False, "Partida sem pontuação: registro não realizado."

    if not 0 <= total_acertos <= total_questoes:
        return False, "Quantidade de acertos inválida."

    # Requisito: registrar data e hora da partida (hora local, como na API)
    conexao.execute(
        """INSERT INTO Partidas (Nickname, Pontuacao, Modo, TotalQuestoes,
                                 TotalAcertos, TempoSegundos, DicasUsadas, DataHora)
           VALUES (?, ?, ?, ?, ?, ?, ?, datetime('now', 'localtime'))""",
        (nickname, pontuacao, modo, total_questoes, total_acertos,
         tempo_segundos, dicas_usadas),
    )
    conexao.commit()
    return True, "Partida registrada."


def consultar_historico(conexao, limite=20):
    """Pós-condição: retorna as partidas do mais recente para o mais antigo."""
    limite = max(1, min(limite, 200))
    cursor = conexao.execute(
        """SELECT Id, Nickname, Pontuacao, Modo, TotalQuestoes, TotalAcertos,
                  TempoSegundos, DicasUsadas, DataHora
           FROM Partidas
           ORDER BY DataHora DESC, Id DESC
           LIMIT ?""",
        (limite,),
    )
    return [dict(linha) for linha in cursor.fetchall()]


if __name__ == "__main__":
    con = conectar()
    print(registrar_partida(con, "Ana", 140, "Quiz", 10, 10, 7, 112, 2))
    print(registrar_partida(con, "", 60, "", 10, 10, 4, 150, 0))
    print(registrar_partida(con, "Joao", 30, "Quiz", 10, 8, 3, 90, 1))
    print(registrar_partida(con, "Bia", 0, "Quiz", 10, 10, 0, 200, 0))
    for partida in consultar_historico(con):
        print(partida)
```

Exemplo de execução: o código foi testado em uma cópia do banco oficial quiz.db. As duas primeiras partidas são registradas (a segunda como “Anônimo” e no modo padrão “Quiz”), a terceira é recusada por estar incompleta e a quarta por não ter pontuação. A consulta mostra as partidas registradas, da mais recente para a mais antiga:

```
(True, 'Partida registrada.')
(True, 'Partida registrada.')
(False, 'Partida incompleta: nem todas as questões foram respondidas.')
(False, 'Partida sem pontuação: registro não realizado.')
{'Id': 2, 'Nickname': 'Anônimo', 'Pontuacao': 60, 'Modo': 'Quiz', 'TotalQuestoes': 10, ...}
{'Id': 1, 'Nickname': 'Ana', 'Pontuacao': 140, 'Modo': 'Quiz', 'TotalQuestoes': 10, ...}
```

5.2.3 Implementação no sistema oficial (C#)

No sistema oficial, a mesma rotina está implementada na QuizArcorApi, em C#. O endpoint POST /api/partidas recebe o resultado enviado pelo aplicativo, aplica as regras de nickname, modo e pontuação e grava a partida:

```
// ---------- POST /api/partidas ----------
app.MapPost("/api/partidas", (PartidaDto p) =>
{
    if (p.Pontuacao < 0)
        return Results.BadRequest(new { erro = "Pontuação inválida." });

    string nick = string.IsNullOrWhiteSpace(p.Nickname) ? "Anônimo" : p.Nickname.Trim();
    string modo = string.IsNullOrWhiteSpace(p.Modo) ? "Quiz" : p.Modo.Trim();

    using var cn = Abrir();
    using var cmd = cn.CreateCommand();
    cmd.CommandText = @"
        INSERT INTO Partidas (Nickname, Pontuacao, Modo, TotalQuestoes, TotalAcertos,
TempoSegundos, DicasUsadas, DataHora)
        VALUES ($nick, $pont, $modo, $q, $a, $t, $d, datetime('now', 'localtime'));
        SELECT last_insert_rowid();";
    cmd.Parameters.AddWithValue("$nick", nick);
    cmd.Parameters.AddWithValue("$pont", p.Pontuacao);
    cmd.Parameters.AddWithValue("$modo", modo);
    cmd.Parameters.AddWithValue("$q", p.TotalQuestoes);
    cmd.Parameters.AddWithValue("$a", p.TotalAcertos);
    cmd.Parameters.AddWithValue("$t", p.TempoPartida);
    cmd.Parameters.AddWithValue("$d", p.DicasUsadas);

    long id = (long)cmd.ExecuteScalar()!;
    return Results.Created($"/api/partidas/{id}", new { id, mensagem = "Partida
registrada." });
});
```

O endpoint GET /api/partidas faz a consulta do histórico, do mais recente para o mais antigo:

```
// ---------- GET /api/partidas?limite=20 (historico) ----------
app.MapGet("/api/partidas", (int? limite) =>
{
    using var cn = Abrir();
    using var cmd = cn.CreateCommand();
    cmd.CommandText = @"
        SELECT Id, Nickname, Pontuacao, Modo, TotalQuestoes, TotalAcertos, TempoSegundos,
DicasUsadas, DataHora
        FROM Partidas
        ORDER BY DataHora DESC, Id DESC
        LIMIT $limite";
    cmd.Parameters.AddWithValue("$limite", Math.Clamp(limite ?? 20, 1, 200));

    ...
```

Na versão em C#, a regra “registrar apenas se todas as questões forem respondidas” é garantida pelo próprio fluxo do aplicativo, que só envia o resultado depois da última pergunta. Já a regra “se não há pontuação, não fazer o registro” está presente na versão em Python, mas ainda não no endpoint em C#, que hoje aceita partidas com pontuação zero.

6 ARQUITETURA DO SISTEMA

Esta seção descreve a arquitetura da versão oficial do Quiz Arcor Challenge, da camada de apresentação até a persistência de dados, com foco em modularidade, simplicidade de manutenção e eficiência.

6.1 Visão geral do sistema

- **Nome do sistema:** Quiz Arcor Challenge (na tela do jogo: “Arcor – Desafio das Marcas”).
- **Objetivo do sistema:** oferecer um quiz interativo sobre a Arcor e as marcas Tortuguita e Block, que transforma informações da marca em uma experiência divertida, com pontuação por dificuldade, pistas, tempo e feedback educativo.
- **Público-alvo:** adolescentes e adultos, com foco na Geração Z (Block) e nos millennials (Tortuguita), além da equipe responsável por manter as perguntas.
- **Principais funcionalidades:**
  - tela inicial com instruções (“Como jogar”) e identificação por apelido ou modo anônimo;
  - escolha da dificuldade (Todas, Fácil, Médio ou Difícil);
  - partida de 10 perguntas sorteadas, com 20 segundos por pergunta, pista e feedback;

- cálculo da pontuação, classificação e resumo da partida;
- histórico das últimas partidas;
- conteúdo mantido no banco de dados, com possibilidade de inativar perguntas e marcas.

6.2 Arquitetura geral

O sistema adota a **arquitetura em camadas** (Layered Architecture), no modelo cliente-servidor. O cliente é um aplicativo desktop em C# com Windows Forms (projeto QuizArcor), que se comunica por HTTP/JSON com uma API REST em ASP.NET Core (projeto QuizArcorApi). A API concentra o acesso aos dados e grava as informações em um banco SQLite. Essa separação permite alterar uma camada sem afetar as demais: por exemplo, trocar o visual das telas sem mexer na API, ou cadastrar novas perguntas sem alterar o código.

6.3 Componentes arquiteturais e camadas

6.3.1 Camada de apresentação (frontend)

- **Tecnologias:** C# com Windows Forms, .NET 10 (net10.0-windows).
- **Componentes:** Form1.cs, com cinco painéis (Início, Identificação, Quiz, Resultado e Histórico); Visual.cs, com as cores do tema e os controles desenhados (PainelImagem, PainelCartao, BotaoJogo e BarraTempo); e a pasta Imagens.
- **Responsabilidades:**
  - interface com o usuário, seguindo o protótipo da seção 2.4;
  - validação inicial de dados: apelido ou anônimo, descarte de perguntas inconsistentes e validação da partida (CalculadoraPartida.Validar);
  - controle da partida: tempo de 20 segundos, pista, atalhos de teclado e feedback visual;
  - comunicação com a API via HTTP/JSON, pela classe QuizApiService (HttpClient com endereço http://localhost:5000 e tempo limite de 10 segundos).

6.3.2 Camada de aplicação (backend/API)

- **Tecnologias:** C# com ASP.NET Core Minimal API, .NET 10, e o pacote Microsoft.Data.Sqlite (versão 10.0.13). A rotina de Histórico também foi codificada em Python (seção 5.2), conforme pedido no template da especificação.
- **Responsabilidades:**
  - exposição de endpoints RESTful, conforme o Quadro 3;
  - comunicação com a camada de dados por consultas SQL parametrizadas;

- validação dos dados recebidos (pontuação não negativa, nickname e modo padrão);
- preparação do banco ao iniciar (PrepararBanco), conferindo as tabelas e acrescentando colunas que faltarem;
- tratamento de erros: falhas inesperadas viram uma resposta JSON, sem derrubar a API.

- **Autenticação/autorização:** não há login. O jogador é identificado apenas pelo apelido ou como anônimo, e a API atende somente o próprio computador (localhost).

Quadro 3 – Endpoints da API QuizArcorApi

Método e rota	Finalidade	Requisito
GET /	Confirmar que a API está no ar	–
GET /api/marcas	Listar as marcas ativas (Tortuguita e Block)	RFS02
GET /api/perguntas?quantidade=&dificuldade=&marcaId=&excluir=	Sortear perguntas ativas com alternativas, pista, feedback e marca	RFS02, RFS03
POST /api/partidas	Registrar o resultado da partida no histórico	RFS06
GET /api/partidas?limite=	Consultar o histórico (mais recentes primeiro)	RFS06

Fonte: Elaborado pelos autores (2026).

6.3.3 Camada de domínio (negócio)

- **Descrição:**
  - contém as regras de negócio e os modelos de domínio do Quiz, no arquivo Partida.cs;
  - as regras de cálculo (CalculadoraPartida) não dependem das telas nem do banco de dados.
- **Componentes:**
  - entidades e modelos: Pergunta, Alternativa, DadosPartida, ResultadoPartida e PartidaHistorico no aplicativo, e PartidaDto na API. O modelo conceitual completo está no diagrama de classe da seção 4.2;
  - casos de uso: Ver instruções, Informar nickname, Escolher modo de jogo, Jogar Quiz, Ver feedback educativo e Ver resultado final (seção 4.1);
  - regras de negócio: 10, 20 ou 30 pontos por acerto conforme a dificuldade; 5 pontos de desconto por pista; pontuação mínima zero; tempo esgotado conta como erro; classificação Mestre das Marcas, Conhecedor de Marcas, Aprendiz ou Iniciante; ritmo e melhor nível.

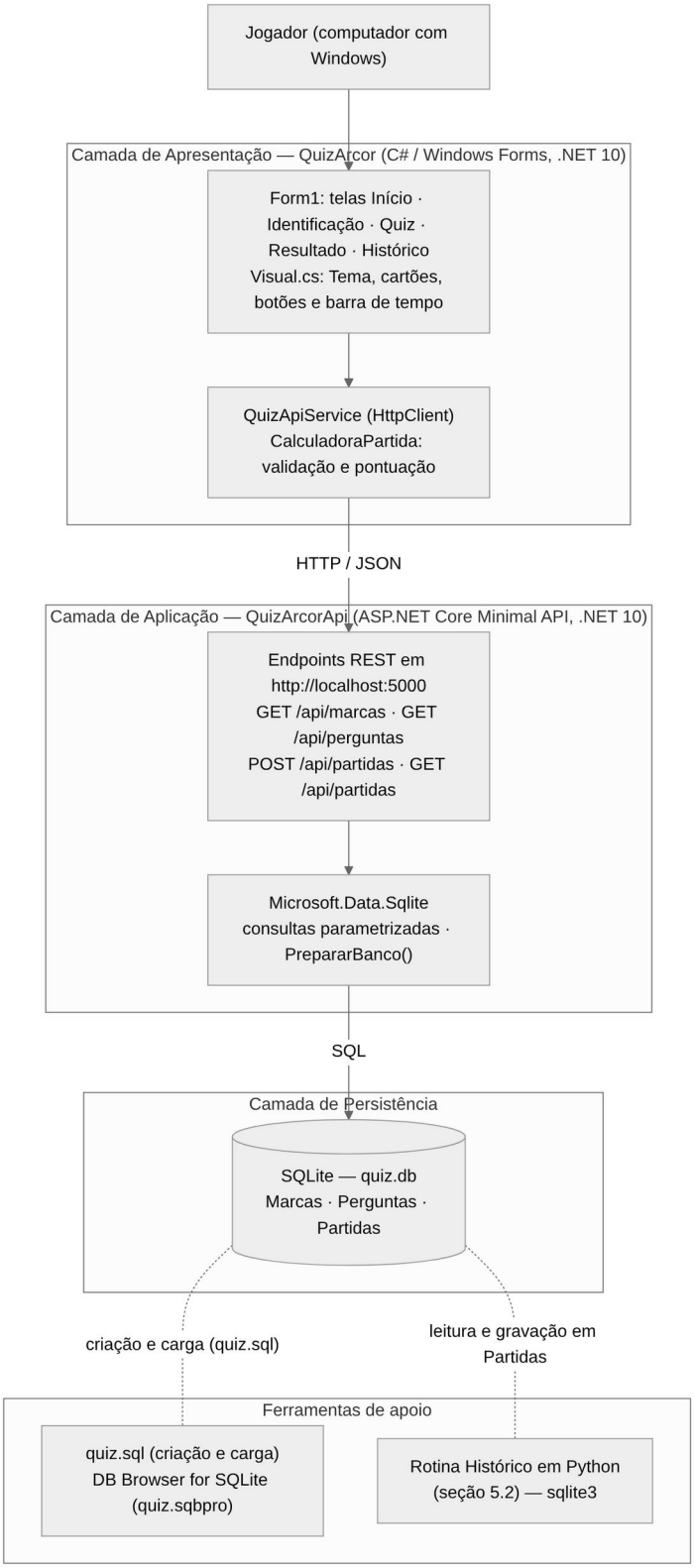
6.3.4 Camada de persistência (banco de dados)

- **Tecnologia:** SQLite, em um arquivo único (quiz.db) na pasta da API. O caminho pode ser alterado pela configuração “Banco”.
- **Responsabilidades:**
  - armazenamento e recuperação de dados nas tabelas Marcas (Id, NomeMarca, Descricao, Ativo), Perguntas (Id, Enunciado, AlternativaA a AlternativaD, RespostaCorreta, Dificuldade, Pista, Feedback, MarcaId, Ativo) e Partidas (Id, Nickname, Pontuacao, DataHora, Modo, TotalQuestoes, TotalAcertos, TempoSegundos, DicasUsadas);
  - script de criação e carga quiz.sql, que cria as tabelas e insere as marcas e as perguntas (a versão atual tem 30 perguntas, 10 de cada dificuldade), executado no DB Browser for SQLite (projeto quiz.sqbpro);
  - migração simples: ao iniciar, a API acrescenta as colunas novas (Ativo e as métricas de Partidas) se o banco for de uma versão anterior.

6.4 Diagrama da arquitetura

A Figura 11 mostra o fluxo entre o jogador, o aplicativo, a API e o banco de dados, além das ferramentas de apoio.

Figura 11 – Diagrama da arquitetura do Quiz Arcor Challenge



Fonte: Elaborado pelos autores (2026).

6.5 Organização de pastas e módulos

O código do projeto está organizado no repositório da seguinte forma, seguindo o caminho usado na especificação do Histórico (Projeto6\src\Entrega 2\Backend\QuizArcorApi):

```
Projeto6/
└── src/
    └── Entrega 2/
        ├── Backend/
        │   └── QuizArcorApi/
        │       ├── Program.cs                (endpoints, PrepararBanco)
        │       ├── QuizArcorApi.csproj       (net10.0 + Microsoft.Data.Sqlite)
        │       ├── appsettings.json
        │       ├── appsettings.Development.json
        │       ├── Properties/
        │       │   └── launchSettings.json
        │       ├── quiz.db                   (banco SQLite)
        │       ├── quiz.sql                  (criação das tabelas e carga)
        │       └── quiz.sqbpro               (projeto do DB Browser for SQLite)
        ├── Frontend/
        │   └── QuizArcor/
        │       ├── Program.cs                (ponto de entrada do aplicativo)
        │       ├── Form1.cs                  (telas e fluxo da partida)
        │       ├── Partida.cs                (modelos, CalculadoraPartida,
        │       │                              QuizApiService)
        │       ├── Visual.cs                 (Tema, painéis, botões, barra)
        │       ├── QuizArcor.csproj          (net10.0-windows, Windows Forms)
        │       └── Imagens/                  (capa, fundos, mascotes,
        │                                      troféu, Tortuguita e Block)
        └── Rotinas/
            └── historico.py                  (rotina da seção 5.2)
```

6.6 Integrações externas

- **APIs de terceiros:** não há. O sistema usa apenas a própria API (QuizArcorApi).
- **Serviços de autenticação:** não há. O jogador é identificado por apelido ou como anônimo, sem OAuth2, Firebase ou senha.
- **Serviços de envio:** não há envio de e-mail ou notificações nesta versão. O compartilhamento do resultado e o ranking entre jogadores ficam como evoluções futuras, e o ranking pode ser montado a partir da tabela Partidas.

6.7 Ambiente e implantação

- **Ambientes:** desenvolvimento local nas máquinas da equipe (Visual Studio ou VS Code com o .NET 10 SDK) e testes do grupo antes da entrega. A apresentação do PI é feita no próprio computador.

- **Execução:** primeiro, rodar o script quiz.sql no arquivo quiz.db (DB Browser for SQLite); depois, executar “dotnet run” na pasta QuizArcorApi, que fica disponível em http://localhost:5000; por fim, abrir o aplicativo QuizArcor.
- **Contêineres:** não são usados nesta entrega; o uso de Docker fica como evolução futura.
- **CI/CD:** não há pipeline automatizado nesta versão; o código é versionado em repositório Git.
- **Hospedagem:** local (localhost). Como o banco é um arquivo SQLite, a API poderia ser publicada futuramente em um serviço como Azure App Service sem precisar de um servidor de banco separado.

7 REFERÊNCIAS

ARCOR. **Chocolates**: informações institucionais e portfólio. 2026.

HONG, Y.; SAAB, N.; ADMIRAAL, W. Approaches and game elements used to tailor digital gamification for learning: a systematic literature review. **Computers & Education**, 2024.

KANTAR. **Queda no consumo de chocolate em meio à crise do cacau**. 2025.

MENTIMETER. **How to create a quiz competition**. 2026.

NESTLÉ BRASIL. **Chocolates**. 2026.

QUIZIZZ. **Plataforma de aprendizagem gamificada**. 2026.

SAILER, M. et al. How gamification motivates: an experimental study of the effects of specific game design elements on psychological need satisfaction. **Computers in Human Behavior**, v. 69, p. 371-380, 2017.

SOMMERVILLE, I. **Engenharia de software**. 11. ed. São Paulo: Pearson Addison-Wesley, 2017.

WANG, A. I.; TAHIR, R. The effect of using Kahoot! for learning: a literature review. **Computers & Education**, v. 149, 2020.

ZAINUDDIN, Z. et al. The impact of gamification on learning and instruction: a systematic review of empirical evidence. **Educational Research Review**, v. 30, 2020.